

PlayMatch V3

Propuesta Técnica, Arquitectura de Software y Modelo de Negocio Híbrido

PROYECTO

Plataforma Hiperlocal de Conexión Deportiva Ely Saul Chirivella

AUTOR / LEAD DEV

VERSIÓN / FECHA

V3.0 — Septiembre 2026

1. Visión del Producto y Evolución del Modelo

PlayMatch es una ecosistema hiperlocal diseñado para eliminar la fricción operativa en la organización de encuentros deportivos amateurs. La plataforma combina geolocalización en tiempo real, asignación táctica por posiciones en cancha, métricas de confiabilidad de jugadores y reservas inteligentes de recintos deportivos.

Actualización Clave en V3: Modelo Híbrido de Acceso y Monetización

Para garantizar la máxima retención y abarcar tanto a deportistas esporádicos como a jugadores frecuentes, la plataforma se estructura bajo **dos vertientes comerciales integradas**: el modelo tradicional de **Pago Único por Partido (Pay-Per-Match)** y el modelo recurrente de **Suscripción (Membresías / Pases Deportivos)**.

2. Modelo Comercial y Monetización: Las Dos Vertientes

El sistema permite operar de manera fluida entre usuarios que prefieren pagar exclusivamente por cada encuentro en el que participan y usuarios que buscan una tarifa plana con beneficios exclusivos.

Vertiente 1: Pago Único por Partido (Pay-Per-Match)

Perfil del Usuario: Jugador casual, visitante esporádico o usuario nuevo que desea probar la aplicación sin compromisos.

- **Sin Cuota Mensual:** El usuario solo realiza transacciones cuando decide sumarse a un partido.
- **Pago por Cupo (pricePerSlot):** Se cobra el valor individual asignado al encuentro en la pasarela de pagos.
- **Flexibilidad Absoluta:** Ideal para quienes juegan de forma imprevista o en distintas ciudades.
- **Transparencia:** Desglose claro de tarifa de cancha + comisión de servicio de plataforma por partido.

Vertiente 2: Modelo de Suscripción (Membresías / Pases)

Perfil del Usuario: Deportista recurrente (1 a 4 partidos semanales), competidor regular o miembro de ligas.

- **Cuota Periódica:** Planes mensuales o trimestrales (ej. Pase Básico 4 partidos/mes, Pase PRO Ilimitado).
- **Ahorro Económico:** Descuento significativo del 20% al 35% en el costo promedio por partido.
- **Cero Comisiones Extra:** Exoneración total de fees transaccionales individuales.
- **Prioridad de Reserva:** Acceso anticipado a la grilla de partidos y prioridad automática en lista de espera (Waitlist).

Criterio Comparativo	Pago Único por Partido	Suscripción (Membresía)
Compromiso Financiero	Ninguno (Transaccional según demanda).	Recurrente (Cobro mensual / trimestral automático).
Mecanismo de Verificación	Pasarela de pago directa (Stripe / Pago Móvil / Tarjeta).	Débito de créditos del saldo del usuario (UserSubscription).
Selección Táctica de Posición	Habilitada en el momento del pago.	Habilitada de forma prioritaria (reserva anticipada).
Gestión en Lista de Espera	Orden cronológico estándar.	Prioridad VIP sobre usuarios de pago único.

3. Arquitectura de Permisos y Roles (RBAC Ampliado)

El control de acceso basado en roles garantiza la segregación de funciones entre los jugadores, administradores de instalaciones y moderadores de la plataforma:

- PLAYER Jugador Estándar (Pay-Per-Match):** Explora el feed geolocalizado, se une a partidos mediante pago puntual por cupo, elige posición en cancha y califica el comportamiento/asistencia de sus compañeros.
- PLAYER_SUBSCRIBER Jugador Suscriptor:** Cuenta con una membresía activa (`isActive = true``), lo que le permite inscribirse utilizando créditos acumulados sin pasar por pasarela de pago individual.
- COURT_MANAGER Administrador de Canchas:** Perfil B2B. Registra recintos deportivos con ubicación PostGIS, administra horarios, gestiona reservas y recibe la liquidación tanto de pagos individuales como de cupos cubiertos por suscriptores.
- SUPER_ADMIN Administrador del Sistema:** Control total del backoffice, moderación de usuarios, configuración de planes de suscripción, gestión de disputas y analíticas financieras.

4. Modelo Relacional de Base de Datos (Prisma Schema V3)

Esquema diseñado para PostgreSQL con extensión espacial PostGIS, integrando soporte nativo para el modelo dual de pagos y gestión de suscripciones:

```
// Prisma Schema - PlayMatch V3 (PostgreSQL + PostGIS)
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

enum Role {
  SUPER_ADMIN
  COURT_MANAGER
  PLAYER
}

enum PaymentType {
  PAY_PER_MATCH
  SUBSCRIPTION_CREDIT
}

enum SubscriptionTier {
  BASIC_PASS      // Ej. 4 partidos al mes
  PRO_MONTHLY     // Ej. 8 partidos al mes + Prioridad
  VIP_ANNUAL      // Partidos ilimitados
}

model User {
  id          String      @id @default(uuid())
  name        String
  email       String      @unique
  role        Role        @default(PLAYER)
  skillLevel  Int          // 1 a 5 estrellas
  reliability  Float       // % Histórico de asistencia
  location     Unsupported("geometry(Point, 4326)")?
  matchesHosted Match[]    @relation("Host")
  enrollments  Participant[]
  courtsManaged Court[]
  subscriptions UserSubscription[]
  transactions Transaction[]
}

model SubscriptionPlan {
  id          String      @id @default(uuid())
  name        String
  tier         SubscriptionTier
  price       Float
  matchesIncluded Int      // Límite mensual de créditos
  priorityBooking Boolean  @default(true)
  userSubscriptions UserSubscription[]
}

model UserSubscription {
  id          String      @id @default(uuid())
  userId      String
  planId      String
  startDate   DateTime    @default(now())
  endDate     DateTime
  isActive    Boolean     @default(true)
  remainingMatches Int     // Saldo de créditos disponibles
  user        User        @relation(fields: [userId], references: [id])
  plan        SubscriptionPlan @relation(fields: [planId], references: [id])
}

model Match {
  id          String      @id @default(uuid())
  title       String
  sport       String      // Fútbol 5, Fútbol 7, Pádel, Básquet
  scheduledAt DateTime
  durationMins Int        @default(60)
  capacity    Int         // Cupos totales
  pricePerSlot Float      // Costo individual para Pago Único
}
```

```

allowSubscription Boolean      @default(true)
status             String      // OPEN, FULL, IN_PROGRESS, COMPLETED
hostId             String
host              User         @relation("Host", fields: [hostId], references: [id])
courtId            String
court             Court        @relation(fields: [courtId], references: [id])
participants       Participant[]
}

model Participant {
  id                String      @id @default(uuid())
  userId            String
  matchId           String
  status            String      // CONFIRMED, WAITLIST
  position          String?     // Ej. Portero, Defensa, Delantero
  paymentType       PaymentType // PAY_PER_MATCH o SUBSCRIPTION_CREDIT
  user              User        @relation(fields: [userId], references: [id])
  match             Match       @relation(fields: [matchId], references: [id])

  @@unique([userId, matchId])
}

model Transaction {
  id                String      @id @default(uuid())
  userId            String
  amount            Float
  type              PaymentType
  status            String      // COMPLETED, REFUNDED, FAILED
  createdAt         DateTime    @default(now())
  user              User        @relation(fields: [userId], references: [id])
}

```

5. Casos de Uso Principales (Flujos de Negocio)

CU-01: Inscripción mediante Pago Único (Pay-Per-Match)

Actor Principal: Usuario con rol PLAYER (sin suscripción activa).

1. El usuario abre la app y explora el feed de partidos cercanos (consulta espacada con PostGIS `ST\_DWithin`).
2. Selecciona un partido abierto y hace clic en "Unirme al Partido".
3. El backend detecta que no posee una suscripción activa ni créditos disponibles.
4. El sistema despliega la pantalla de Checkout mostrando la cuota única del cupo (`pricePerSlot`) y las opciones de pago.
5. Tras completarse la transacción financiera, el sistema crea el registro en `Participant` con `paymentType = PAY\_PER\_MATCH` y le permite asignar su posición táctica en el campo.

CU-02: Inscripción Directa mediante Crédito de Suscripción

Actor Principal: Usuario con rol PLAYER y suscripción activa.

1. El suscriptor selecciona un partido de la grilla recomendada.
2. Al presionar "Reservar con mi Pase", el sistema valida en tiempo real:
 - Vigencia del plan: `endDate >= NOW()` y `isActive = true`.
 - Saldo de créditos: `remainingMatches > 0`.
3. El sistema descuenta 1 crédito del registro `UserSubscription` (`remainingMatches - 1`).
4. Se confirma inmediatamente la plaza reservando la posición táctica deseada sin requerir pasarela de pago.

CU-03: Creación de Partido y Reserva de Cancha (Host)

Actor Principal: Usuario Host / Organizado.

- 1. El organizador define el deporte, fecha, hora y selecciona la cancha en el mapa interactivo.
- 2. Establece los cupos requeridos (ej. 10 jugadores para Fútbol 5) y asigna las posiciones del esquema táctico (1 Portero, 4 Defensas, etc.).
- 3. Fija la cuota individual por jugador (``pricePerSlot``) y activa el toggle de habilitación de pases (``allowSubscription = true``).
- 4. El partido queda publicado en estado ``OPEN`` y se envían notificaciones push hiperlocales a deportistas en un radio de 5km.

6. Roadmap del Producto y Cronograma (12 Semanas)

Fase	Hito Principal	Entregables Clave
Fase 1 (MVP)	Lanzamiento Core	Autenticación RBAC, feed PostGIS, creación de partidos, flujo de Pago Único por partido.
Fase 2	Módulo de Suscripciones & Social	Motor de membresías y créditos, chat en vivo por partido, sistema de confiabilidad de jugadores.
Fase 3	B2B & Panel de Canchas	Dashboard para <code>`COURT_MANAGER`</code> , gestión de horarios, métricas de ocupación y liquidaciones automáticas.
Fase 4	Gamificación & Ligas	Torneos locales, rankings por zona, medallas de rendimiento y estadísticas de juego.

Cronograma de Ejecución Semanal

- **Semanas 1-2 (Diseño & Arquitectura):** Modelado de base de datos en Prisma, flujos UI/UX en Figma, definición de contratos de API REST / WebSockets.
- **Semanas 3-5 (Core Backend & PostGIS):** Implementación de autenticación, consultas espaciales de canchas, lógica de reservas y migraciones PostgreSQL.
- **Semanas 6-8 (Frontend & Módulo Híbrido de Pagos):** Desarrollo de vistas en Next.js/React, integración de Stripe/ Pasarela de pagos, flujo dual de cuotas y suscripciones.
- **Semanas 9-10 (Tiempo Real & Notificaciones):** WebSockets para chat de coordinacion pre-partido, notificaciones Push geolocalizadas y gestión de Waitlist.
- **Semanas 11-12 (QA, Deploy & Cobertura):** Pruebas unitarias/integración, optimización de consultas SQL, despliegue en servidor de producción (Nginx, PM2) y publicación.